

EE144 Lab #2
Localization using Turtlebot4 and Extended
Kalman Filter

Nathan Taing

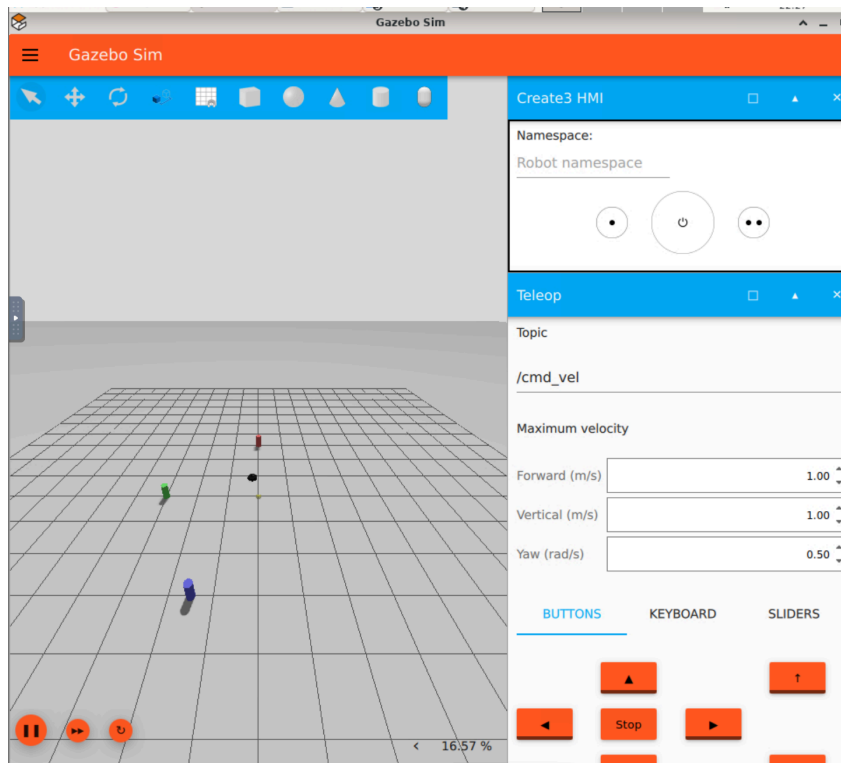
Description

Within this lab, we are working with an Extended Kalman Filter, which aims to understand where it predicts to be vs what it is actually seeing within the environment. A Kalman Filter works in two stages. The first stage takes the information about the robot's motion (i.e. the velocity, turn degrees, angular velocity, etc.) and predicts the state of the robot when it moves to that state. This position is described using a 3x1 matrix describing the x and y position as well as the direction the robot is pointing. The second stage is using the robot's sensors to sense the reality of the position the robot is in. It then tries to calculate the difference and performs the adjustments needed to reduce the error.

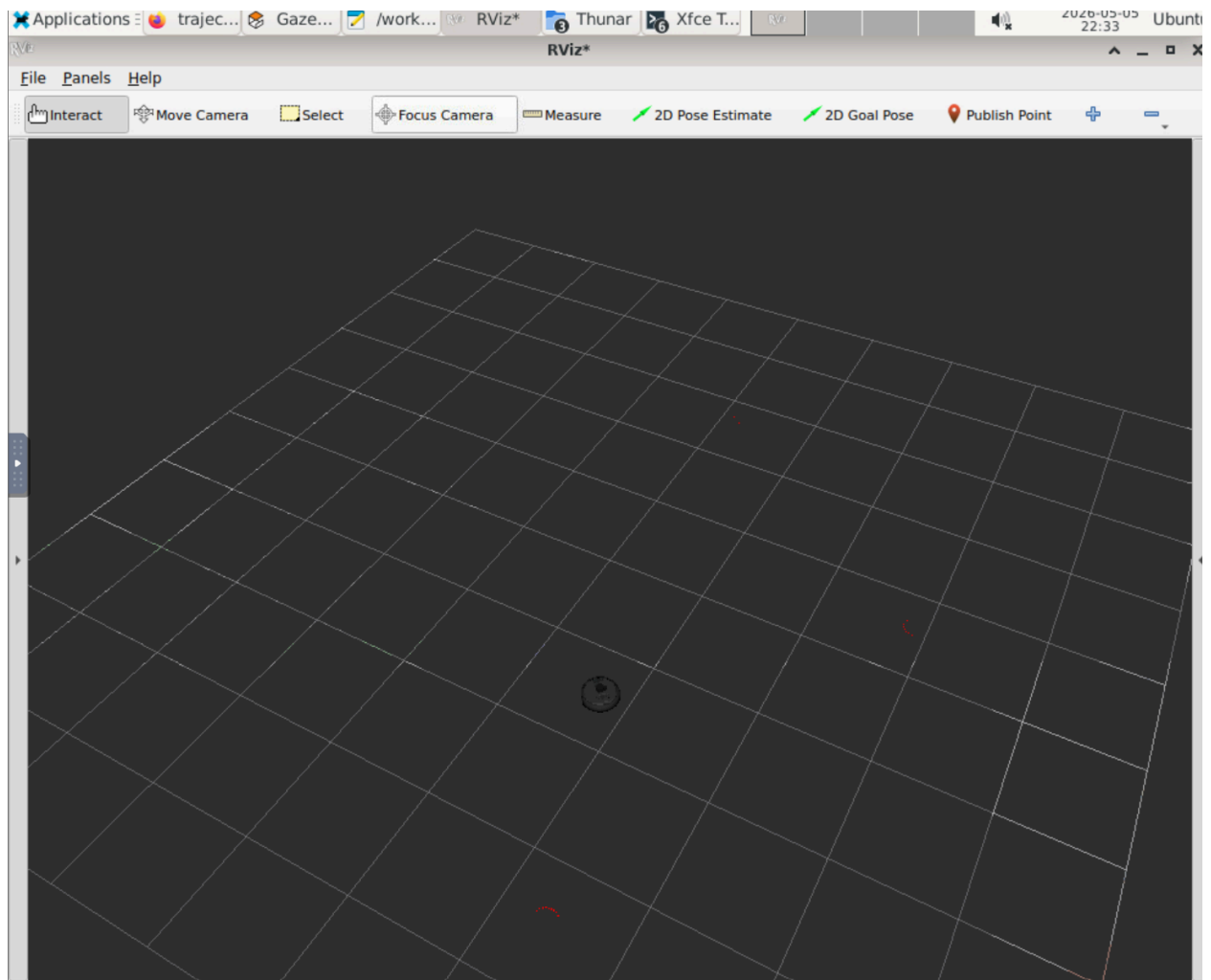
The EKF is trying to minimize the innovation that the LIDAR sensors are detecting, as well as how uncertain we are due to the natural noise of sensors. There is always a natural degree of uncertainty due to measurement equipment not being perfect. Each step of the EKF is trying to bring the projected positioning closer to the actual position. It is important to note that the EKF needs to know the initial map to understand where it should be in relation to all of the detected obstacles. In this lab, we worked on six different equations to help minimize the detected error. They produce matrices to help the robot understand the required corrections and help predict the correct movement for the robot to course correct.

Results

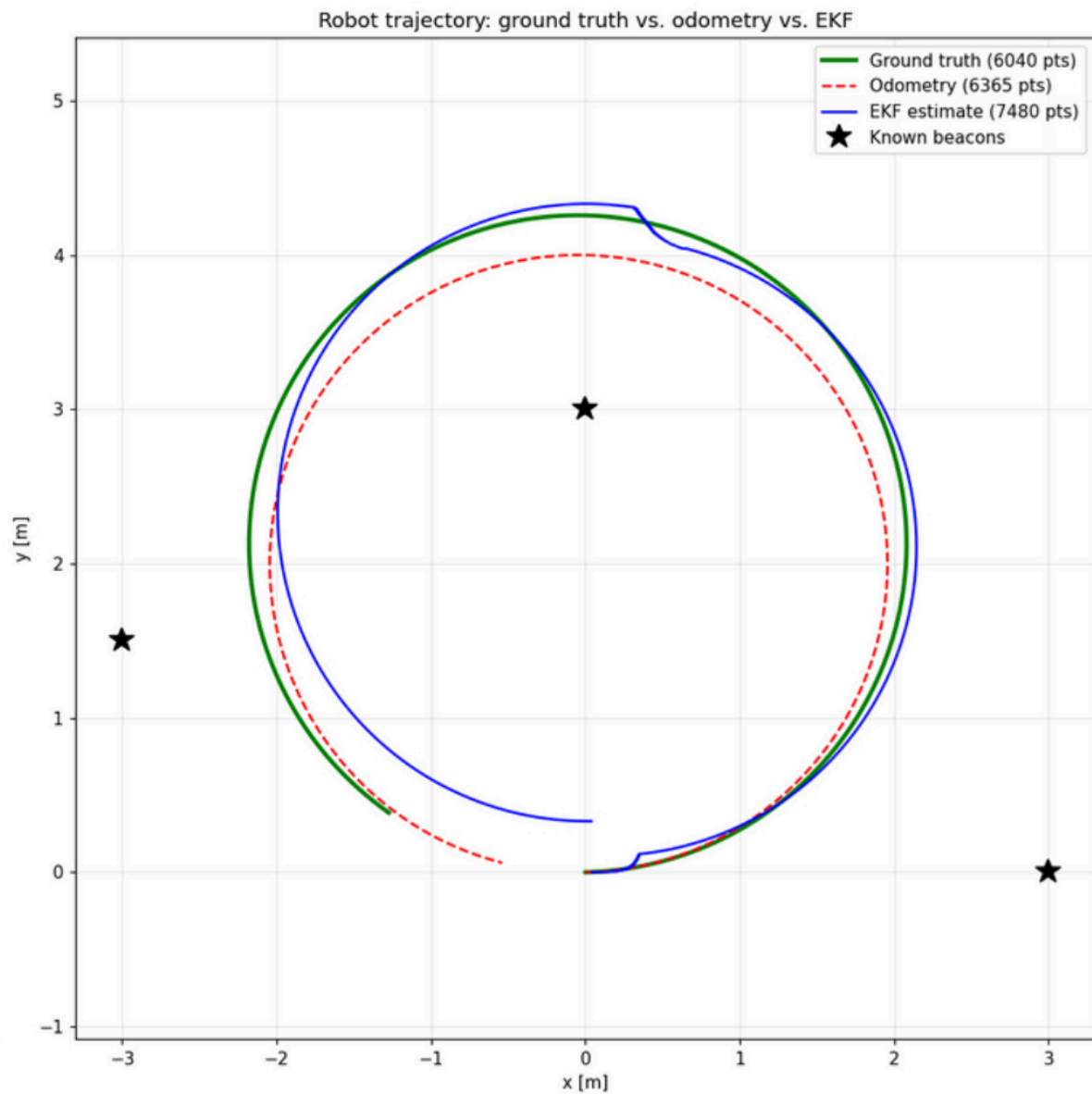
Robot in motion:



Robot with LIDAR:

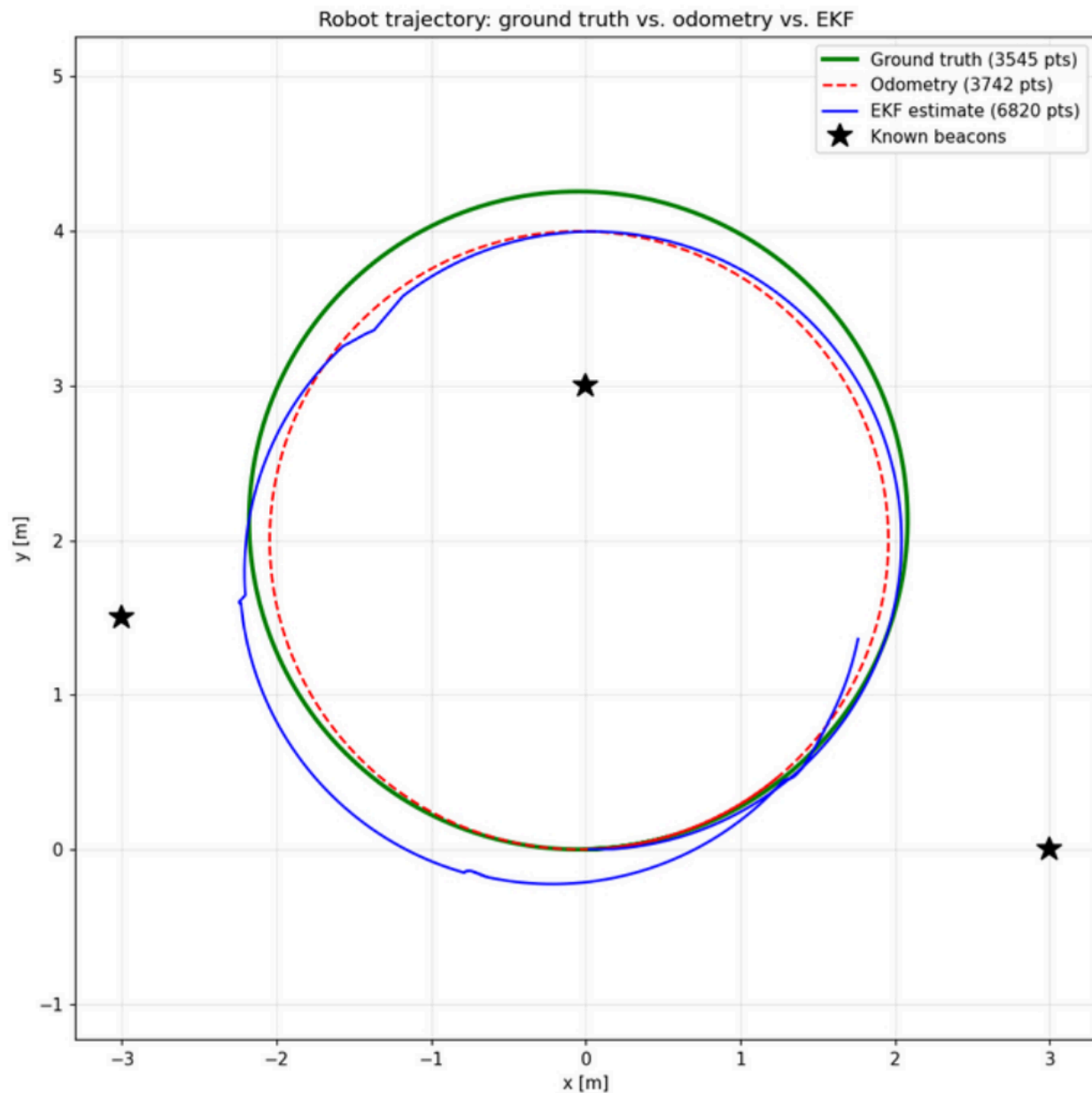


Default Values:



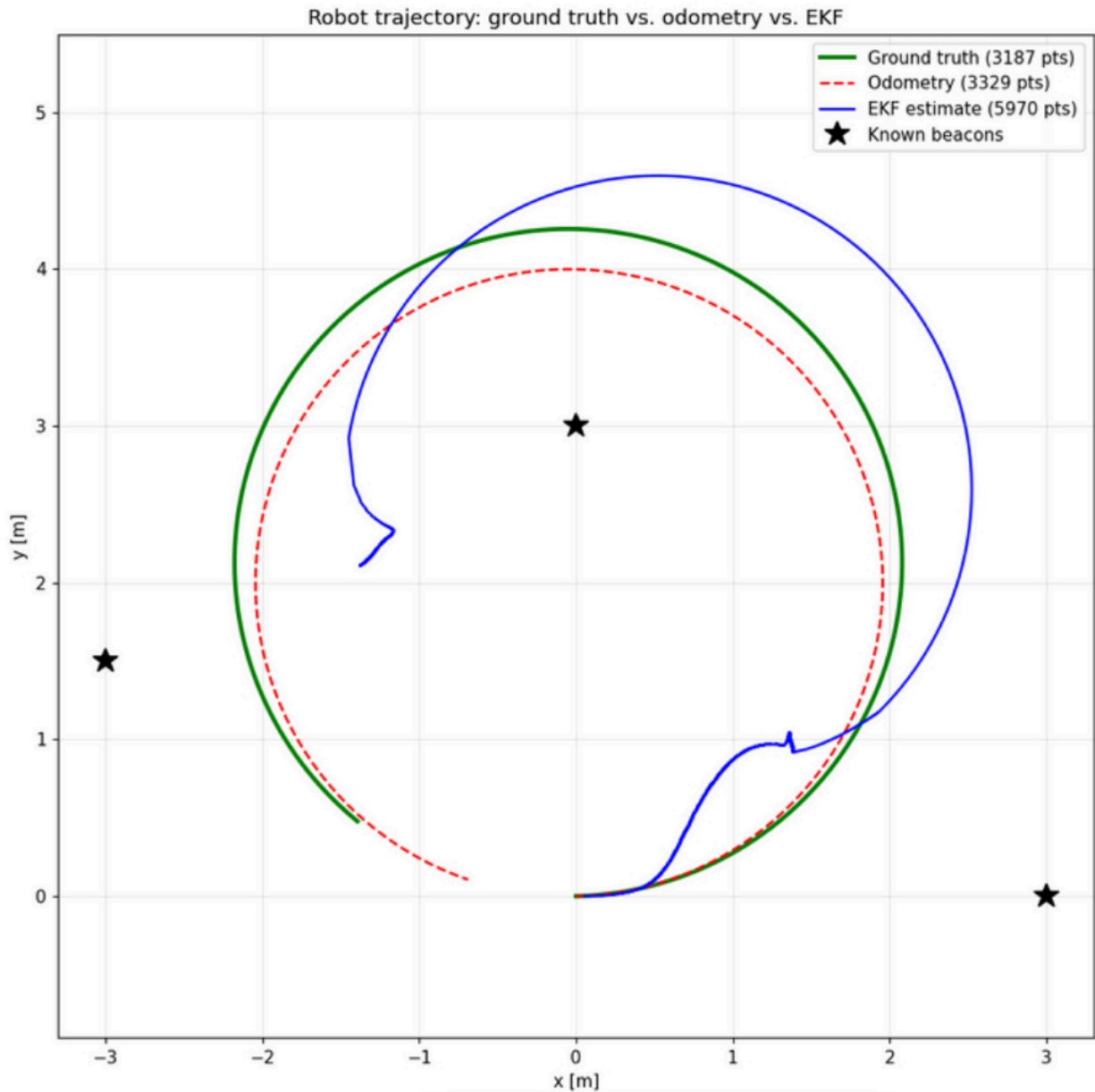
Smaller Values:

SIGMA_V	= 0.01	# m/s	-- linear velocity noise
SIGMA_W	= 0.01	# rad/s	-- angular velocity noise
SIGMA_RANGE	= 0.01	# m	-- LIDAR range noise
SIGMA_BEARING	= 0.01	# rad	-- LIDAR bearing noise



Bigger Values:

SIGMA_V	= 0.1	# m/s	-- linear velocity noise
SIGMA_W	= 0.1	# rad/s	-- angular velocity noise
SIGMA_RANGE	= 0.20	# m	-- LIDAR range noise
SIGMA_BEARING	= 0.1	# rad	-- LIDAR bearing noise



Above, we can see three different test runs with different noise values. According to our data, the smaller the noise value, the tighter our EKF estimate is in relation to where the Odometry is. It trusts the measurement devices more, leading to a closer path of following the dotted line. Adjustments were made quickly to correct back to the Odometry path. The bigger the noise value became, the less filtering the EKF performed. It trusted the Odometry less, leading to the creation of its own path. As we can see in the last picture, our EKF estimate is nowhere close to the true position of the robot. This suggests that noise heavily influences the trajectory of the robot, leading to undercorrection or overreaction. If the noise value is too small, the filter may overreact to imperfect data. If the values are too large, they may not filter enough, causing the shape to drift.

Discussion

The EKF estimate does generally stay closer to ground truth than the odometry estimate. This is because it utilizes the measurement data and compares the real values to the predicted value and aims to correct itself. During the trajectory the biggest difference between the odometry and EKF was towards the end. This is due to the accumulated uncertainty throughout the run and causing the robot to use more miscalculated data. The longer the run, the more uncertainty the robot will have in its measured position. The covariance ellipse around the EKF estimate represents the uncertainty in the robot's position. The diagonal entries of the Sigma matrix represent the variance in the x and y directions, which help determine the shape of the ellipse. A larger ellipse means greater uncertainty and a smaller ellipse indicates a higher trust in the estimated position.